

基于傅里叶变换的频域反演半导体外延层测厚方法

摘要

针对问题一，我们建立了双光束干涉模型。首先，根据几何光学关系和斯涅尔定律，推导了外延层厚度与入射角、折射率、波数之间的数学关系，并得到厚度的解析公式。在此基础上，明确了厚度反演的物理机制，即条纹主频与厚度之间的一一对应关系。该部分为后续算法设计奠定了理论基础。

针对问题二，我们提出了**频域反演算法**。首先对附件 1 和附件 2 中不同入射角下的反射率光谱进行预处理，包括区间裁切、等距重采样、去趋势与去均值，以削弱背景和噪声干扰。随后采用**快速傅里叶变换**提取全谱主频，并在滑动窗口中进行**局部 FFT 计算**，结合抛物线插值实现亚像素级峰值定位。再结合折射率的局部拟合结果，将主频逐点回代为厚度曲线。最后，利用中位数与中位绝对偏差（MAD）进行稳健统计，得到 10° 和 15° 下的厚度结果分别约为 $7.92\ \mu\text{m}$ 与 $7.87\ \mu\text{m}$ ，置信区间高度重叠，双角度差异小于 1%，表明该方法在实验条件下具有较高的可靠性和鲁棒性。

针对问题三，我们建立了多光束干涉模型。通过振幅几何级数推导出总反射率的闭式表达，并结合 **Poisson 核展开**得到多阶谐波分量，进而给出多光束干涉的必要条件，即界面反射率乘积超过一定阈值且条纹精细度达到 1~2 以上时，多束效应显著。进一步对附件 3 和附件 4 的硅样品数据进行 FFT 分析，结果显示主峰及其二倍谐波清晰可见，符合多光束干涉特征。由主峰回代计算，得到厚度在 10° 和 15° 下分别为 $3.66\ \mu\text{m}$ 和 $3.64\ \mu\text{m}$ ，相对差异仅 0.53%。同时对附件 1 和 2 的碳化硅数据进行谐波检测，未发现明显谐波分量，说明 SiC 样品中多光束效应不显著。综上所述，在存在多束干涉时依然可通过主频稳健反演厚度，并利用谐波作为辅助诊断，提高结果的可靠性。

综上，本文通过双光束理论建模、频域反演算法及多光束扩展分析，提出了一套完整的外延层厚度测量方法。该方法不仅在 SiC 样品上实现了 $7.9\sim 8.0\ \mu\text{m}$ 的稳定厚度估计，在硅样品的多束干涉环境下亦保持了一致性。研究结果表明，该方法在物理合理性与数值稳定性方面均具有优势，为碳化硅外延层无损测量提供了可行的解决方案。

关键字： 多光束干涉 频域反演 快速傅里叶变换 无损测厚 **Poisson 核展开**

一、问题重述

1.1 问题背景

碳化硅（SiC）作为第三代半导体材料，因高击穿电场、高热导率等优点，在新能源和高功率电子器件中应用广泛。与传统硅功率器件制作工艺不同, SiC 功率器件的制备需建立在具有一定掺杂浓度的同质外延漂移层上, 因此 SiC 外延生长技术是 SiC 功率器件制备的核心技术之一, 外延质量和缺陷率将直接影响 SiC 器件的性能和成品率。碳化硅外延层厚度是影响器件性能的关键参数, 需实现精准测量。红外干涉法因无损性而被广泛采用, 其原理是通过红外光在外延层与衬底界面产生的反射光形成的干涉条纹的光谱特征可反推出外延层厚度。然而, 外延层折射率受红外光谱波长和掺杂载流子浓度影响, 且可能存在多光束干涉效应, 传统模型往往面临精度不足的挑战。为此, 建立科学可靠的数学模型和算法, 提升外延层厚度测量的准确性与可靠性, 不仅有助于提升碳化硅外延材料的制备水平, 也对推动半导体产业的技术进步具有重要价值。本文旨在通过数学建模方法, 系统研究红外干涉法中的光学干涉机制, 构建厚度计算模型, 并利用实测数据验证其有效性与鲁棒性。

1.2 问题提出

基于上述背景, 要求建立数学模型解决下列问题。

1.2.1 问题一

考虑外延层和衬底界面只发生一次反射、透射所产生的干涉条纹, 两束光叠加形成干涉条纹这种最基本的情形, 如图1所示, 并建立确定外延层厚度的数学模型。

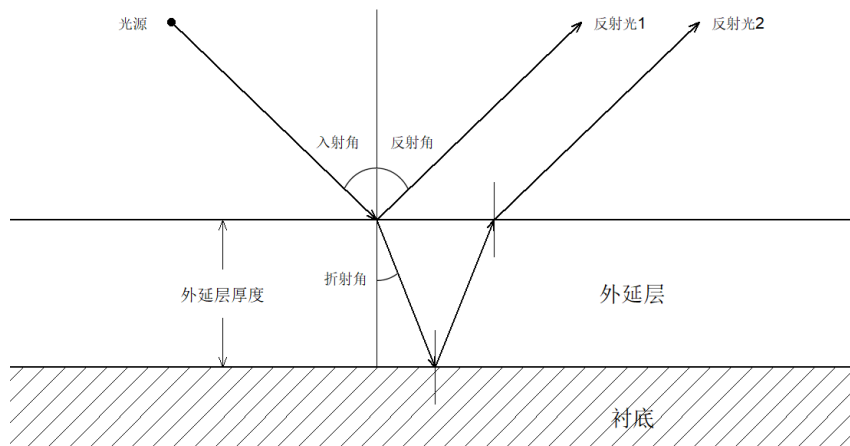


图 1 外延层厚度测量原理示意图

1.2.2 问题二

基于问题一已经建立的单次反射与透射的数学模型，设计能够确定外延层厚度的算法，并根据附件 1 和附件 2 中提供的碳化硅晶圆片在不同入射角（ 10° 和 15° ）下的光谱实测数据（波数与反射率），计算外延层厚度并分析结果的可靠性。

1.2.3 问题三

在更一般的情形下，光波在外延层与衬底界面之间可能发生多次反射和透射，从而形成多光束干涉，如图2所示，这种现象会使干涉条纹更加复杂。因此，本问题的研究目标是：推导多光束干涉产生的必要条件，考虑多光束干涉对碳化硅外延层测量的潜在影响，并根据多光束干涉的必要条件，建立适用于多光束干涉的计算硅外延层厚度的数学模型与算法，通过计算分析附件 3 和附件 4 中提供的硅晶圆片在不同入射角（ 10° 和 15° ）下的测试数据是否发生多光束干涉。附件 1 和附件 2 中的数据也受多光束干涉影响，给出消除干扰影响的改进策略，并给出消除影响后的计算结果。

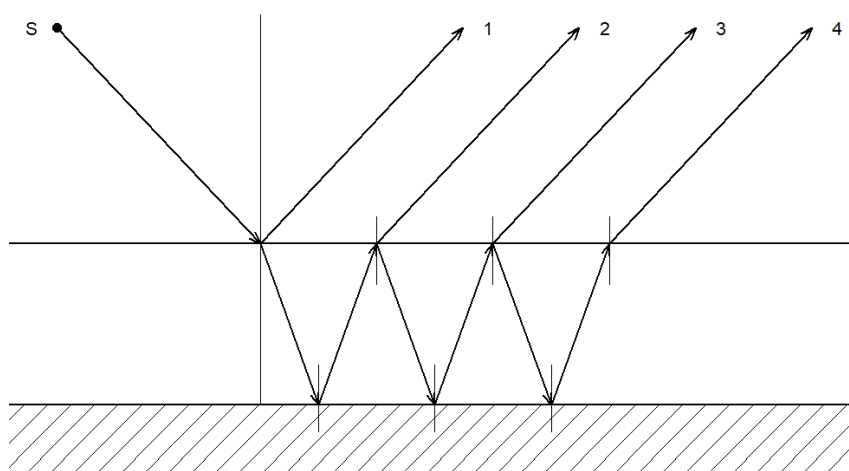


图 2 多光束干涉的示意图

二、问题分析

2.1 问题一的分析

本题首先聚焦于最基本的双光束干涉情形：入射红外光在空气—外延层界面与外延层—衬底界面各发生一次反射与透射，由此形成两束能够相干的出射光。对厚度的识别，本质上依赖于两束光之间稳定且可测的相位差，后者由几何路径与介质色散共同决定。为保证模型具有可操作性并能适配实际数据处理，分析从三方面展开：其一，界面几何与光路结构决定了核心的“有效光程差”，这是最终把条纹周期与外延层厚度联系起来的纽带；其二，斯涅尔定律提供了入射角与折射角之间的约束，使不可直接观测的折射

角可以被消去；其三，干涉极值条件把“可观测的条纹位置/间距”转化为“厚度的代数表达式”。据此，问题一的分析目标是将“几何—物理—可测量量”三者打通：把厚度视为因变量，入射角与谱域自变量（波数/波长）作为观测量，通过明确的推导路径构建可用于数值反演的基础公式。

2.2 问题二的分析

在实验数据层面，反射率光谱通常叠加了慢变背景与仪器响应，同时材料折射率具有显著色散特性，导致条纹不再是严格的等间距正弦。直接在全谱上以固定折射率求厚度，容易产生系统偏差。为兼顾物理准确性与数值稳健性，本部分采用“频域主频提取 + 局部常值近似”的总体思路：先在有效频段内对反射率序列进行等距重采样、去趋势与去均值，以削弱直流与慢变项；随后利用快速傅里叶变换在全谱上获得条纹主频的粗初值，并在滑动窗口内执行局部 FFT，结合峰位亚 bin 插值，逐点估计与波数缓慢共变的“等效主频”。该主频在物理上对应由厚度与入射角共同决定的“等效光程差”，再结合外部给定或拟合得到的折射率色散曲线，能够将主频稳健地回代为厚度曲线。最后，在多角度数据间实施一致性检验和稳健统计（如中位数与 MAD），以评估结果的不确定度与可靠性。

2.3 问题三的分析

在更一般的实际器件与样品中，界面反射并非一次终止；多次往返使得多束光共同参与相干叠加，条纹形状由近似正弦转为具备“尖峰—宽谷”的 Airy 型特征，高阶谐波在频域中可见。多光束是否显著，取决于界面反射率的组合、光源相干长度与薄膜的有效损耗等多因素。就建模层面，需要明确两点：第一，判定条件与可观测证据——例如频谱中是否存在清晰的整数倍谐波峰、峰形是否显著锐化、精细度是否达到可识别阈值；第二，算法层面的兼容性——即在识别多束成立时，如何在不改变厚度——对应关系的前提下，利用主基频稳健反演厚度，并把高阶谐波仅作为“佐证与诊断”而非“额外的未知量来源”。基于此，本部分将提出面向多束情形的判定准则与频域特征提取流程，并给出在存在谐波时仍然以主峰回代厚度的实践策略，同时讨论对色散、噪声与窗口参数的敏感性。

三、模型假设

1. **界面平行且平整假设：**假设外延层的上表面（空气-外延层界面）和下表面（外延层-衬底界面）是光滑、均匀且相互平行的平面。

2. 光源相干长度假设：光源相干长度远大于光程差变化范围，以保证在多次反射后各光束间仍能发生干涉。
3. 材料均匀性假设：假设外延层和衬底材料在各自区域内是光学均匀且各向同性的。
4. 不考虑偏振影响：

s 偏振（电场垂直入射面）：

$$r_s = \frac{\tilde{n}_0 \cos \theta - \tilde{n}_1 \cos \theta'}{\tilde{n}_0 \cos \theta + \tilde{n}_1 \cos \theta'}, \quad t_s = \frac{2\tilde{n}_0 \cos \theta}{\tilde{n}_0 \cos \theta + \tilde{n}_1 \cos \theta'}.$$

p 偏振（电场在入射面）：

$$r_p = \frac{\tilde{n}_1 \cos \theta - \tilde{n}_0 \cos \theta'}{\tilde{n}_1 \cos \theta + \tilde{n}_0 \cos \theta'}, \quad t_p = \frac{2\tilde{n}_0 \cos \theta}{\tilde{n}_1 \cos \theta + \tilde{n}_0 \cos \theta'}.$$

能量反射/透射率（对任一偏振）为

$$R_s = |r_s|^2, \quad R_p = |r_p|^2$$

当法向射入（即 $\theta = 0$ 时），有

$$r_s = r_p = \frac{\tilde{n}_0 - \tilde{n}_1}{\tilde{n}_0 + \tilde{n}_1}, \quad R_s = R_p.$$

本题的入射角度为 $10^\circ, 15^\circ$ ，入射角较小，由

$$\sin \theta \approx \theta, \quad \cos \theta \approx 1 - \frac{1}{2}\theta^2, \quad \cos \theta' \approx 1 - \frac{1}{2}\left(\frac{\tilde{n}_0}{\tilde{n}_1}\right)^2\theta^2,$$

偏振对厚度反演的影响远小于条纹频率定位与噪声带来的不确定度。因此本题目中假设入射角较小，不考虑偏振。

四、符号说明

表 1 符号说明表

符号	说明	单位/量纲
θ	入射角（空气 $\rightarrow$ 外延层）	$^\circ$ 或 rad
θ'	外延层内的折射角	$^\circ$ 或 rad
$n, n(\sigma)$	折射率（随波数的色散）	-
$\tilde{n}_i$	第 i 介质的复折射率	-
$d, d(\sigma)$	外延层厚度（全局/逐点回代）	cm 或 μm
ΔL	有效光程差	cm
σ	波数	cm^{-1}
σ_c	滑动窗口中心的波数	cm^{-1}
λ	波长	cm 或 μm

表格续下页

接上页（符号说明表）

符号	说明	单位/量纲
$\Delta\lambda$	光源谱宽	nm 或 μm
$\Delta\phi(\sigma)$	干涉相位差	rad
δ	相邻往返光束的相位厚度（多束相位步进）	rad
m	干涉级次（整数）	-
k	相位附加项（0 或 1/2）	-
$R(\sigma)$	反射率（随波数）	-
R_i	第 i 个界面的能量反射率	-
R_{total}	多束叠加后的总反射率	-
r_i, t_i	第 i 个界面振幅反射/透射系数（复）	-
r_0, r_1, r_2	空气/膜、膜/衬底、膜内向衬底侧界面振幅反射系数（复）	-
t_0, t_1	空气 $\rightarrow$ 膜、膜 $\rightarrow$ 空气的振幅透射系数（复）	-
E_0	入射到外延层界面的复振幅	V/m
E_i	第 i 束出射反射光的复振幅	V/m
E_r	空气侧总反射复振幅	V/m
A_m	多束 Poisson 展开中第 m 阶谐波幅值系数	-
$S(\theta, \sigma)$	等效主频/等效光程因子 $= 2d\sqrt{n(\sigma)^2 - \sin^2\theta}$	cm
$S(\theta)$	局部常值近似下的等效主频（不显式含 σ ）	cm
S_0	全谱 FFT 主峰的粗初值	cm
$\Delta\sigma$	条纹在波数轴的周期（主峰倒频率）	cm^{-1}
R_s, R_p	s/p 偏振能量反射率	-
r_s, t_s	s 偏振振幅反射/透射系数（复）	-
r_p, t_p	p 偏振振幅反射/透射系数（复）	-
$\text{median}(d)$	厚度的中位数估计（稳健统计）	μm
MAD	中位绝对偏差（稳健离散度）	μm
$d_{10^\circ}, d_{15^\circ}$	$10^\circ/15^\circ$ 入射角下的厚度估计	μm

五、模型建立与求解

5.1 问题一的求解

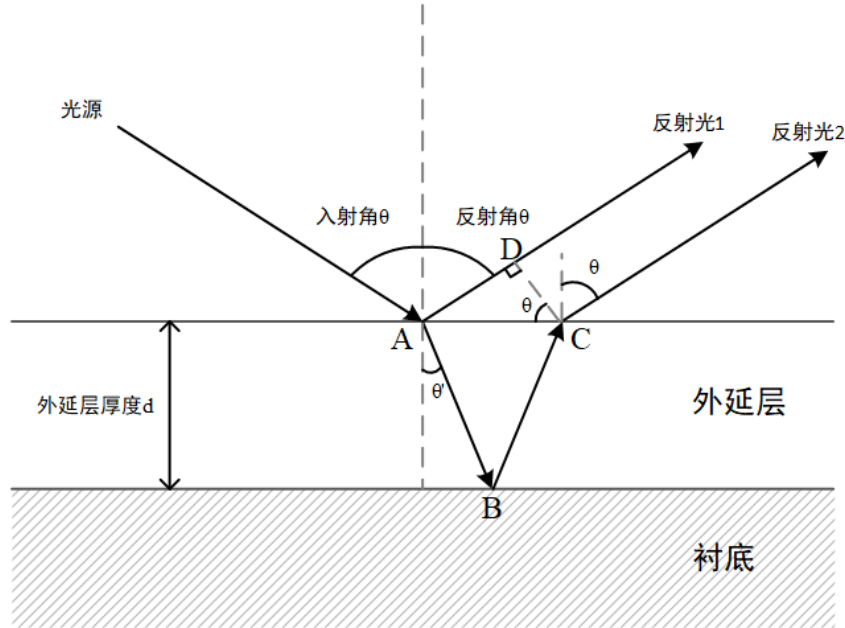


图3 光束干涉模型几何关系示意图

如图3所示，我们建立了外延层和衬底界面只发生一次反射、透射时的光束干涉模型，并分析其中的几何关系。由折射定律我们可以得到第一条关系：

$$n = \frac{\sin \theta}{\sin \theta'}$$

由波数与波长的定义，我们得到第二条关系：

$$\sigma = \frac{1}{\lambda}$$

通过几何分析，我们依次得到以下数量关系：

$$\begin{cases} AD = AC \sin \theta \\ AB = BC = \frac{d}{\cos \theta'} \\ AC = 2d \tan \theta' \\ \Delta L = n(AB + BC) - AD \end{cases}$$

联立以上公式解得：

$$\Delta L = 2d\sqrt{n^2 - \sin^2 \theta}$$

根据光程差与波长的关系，可知：

$$\Delta L = \lambda(m + k) \quad \text{其中, } m \text{ 为整数, } k = 0 \text{ 或 } \frac{1}{2}$$

解得：

$$d = \begin{cases} \frac{m}{2\sigma\sqrt{n^2 - \sin^2 \theta}} & , k = 0 \text{ 时干涉相长} \\ \frac{m + \frac{1}{2}}{2\sigma\sqrt{n^2 - \sin^2 \theta}} & , k = \frac{1}{2} \text{ 时干涉相消} \end{cases}$$

5.2 问题二的求解

由几何到相位的桥接. 由几何光学与折射关系可得有效光程差

$$\Delta L = 2n(\sigma)d \cos \theta' = 2d\sqrt{n(\sigma)^2 - \sin^2 \theta},$$

相位差为

$$\Delta\phi(\sigma) = \frac{2\pi \Delta L}{\lambda} = 2\pi\sigma \Delta L = 4\pi d \sqrt{n(\sigma)^2 - \sin^2 \theta} \sigma.$$

在较窄的波数窗口内, $n(\sigma)$ 可视作局部常值, 反射率可近似写为

$$R(\sigma) \approx E_0(\sigma) + E_1(\sigma) \cos(4\pi d \sqrt{n^2 - \sin^2 \theta} \sigma).$$

定义

$$S(\theta, \sigma) := 2d\sqrt{n(\sigma)^2 - \sin^2 \theta}, \implies \Delta\sigma \approx \frac{1}{S(\theta, \sigma)}.$$

据此, 频域主峰的精确定位与跟踪可转化为对 $S(\theta, \sigma)$ 的估计问题, 进而通过

$$d(\sigma) = \frac{S(\theta, \sigma)}{2\sqrt{n(\sigma)^2 - \sin^2 \theta}}$$

逐点回代得到厚度曲线。

在建立了问题一的理论模型后, 我们尝试将其实测化。附件 1 与附件 2 分别给出了同一块 SiC 晶圆在 10° 与 15° 入射角下的红外反射率光谱。我们的任务是利用这些数据, 结合折射率模型, 反演外延层厚度 d , 并评价结果的可靠性。

观察原始光谱可见, $\sigma < 2000 \text{ cm}^{-1}$ 区间波动剧烈, 受材料吸收峰和实验噪声影响明显, 因此我们选取 $2000 \sim 4000 \text{ cm}^{-1}$ 作为有效区间。随后, 将不等距的波数点插值到等距网格, 并通过多项式去趋势与去均值处理, 消除缓慢变化的背景成分, 使信号近似为

$$R(\sigma) \approx E_0 + E_1 \cos(4\pi d \sqrt{n(\sigma)^2 - \sin^2 \theta} \sigma)$$

由此可知, 干涉条纹的主频率为

$$S(\sigma) = 2d\sqrt{n(\sigma)^2 - \sin^2 \theta}, \quad d(\sigma) = \frac{S(\sigma)}{2\sqrt{n(\sigma)^2 - \sin^2 \theta}}$$

因此我们采取“频域提取 $S(\sigma) \rightarrow$ 回代厚度”的流程。具体方法为：先对全谱进行 FFT, 粗略定位主峰位置 S_0 ; 再以 S_0 为参考做滑动窗口 FFT, 每个窗口包含约五个条纹周期,

步长取 0.5% ~ 1% 周期，从而得到随 σ 平滑变化的 $S(\sigma)$ 曲线。峰位通过抛物线插值做亚 bin 校正，保证分辨率。

折射率 $n(\sigma)$ 取自 Wang 等人在 RefractiveIndex.info 上公布的 4H-SiC 实验数据。在 $2000 \sim 3000 \text{ cm}^{-1}$ 区间内， n 约在 2.44 ~ 2.54 之间缓慢变化。^[2] 为兼顾物理性与计算简便，我们在每个滑窗中心点 σ_c 上对数据库数据做局部线性拟合，得到 $n(\sigma_c)$ ，然后代入上述公式回代厚度。

按照上述方法，我们得到两条厚度曲线 $d_{10^\circ}(\sigma)$ 与 $d_{15^\circ}(\sigma)$ 。将整条曲线看作对同一物理量的重复测量，我们采用中位数作为估计值，用 MAD（中位绝对偏差）衡量离散度。结果如下：

$$10^\circ : \text{median}(d) \approx 7.92 \mu\text{m}, \quad 95\% \text{ 置信区间} \approx [7.80, 8.04] \mu\text{m};$$

$$15^\circ : \text{median}(d) \approx 7.87 \mu\text{m}, \quad 95\% \text{ 置信区间} \approx [7.77, 7.97] \mu\text{m}.$$

两角度计算结果差异仅为 $0.05 \mu\text{m}$ ，相对误差小于 1%，说明算法在不同实验条件下具有较好的稳定性。

如果采用传统方法，在 $\lambda = 5 \mu\text{m}$ 处直接取 $n = 2.5$ 并利用全谱 FFT 主峰计算，则会得到 $d \approx 7.89 \mu\text{m}$ 。

综合两组实验数据与统计分析，我们最终确定该 SiC 外延层的厚度为

$$d \approx 7.9 \mu\text{m}$$

表 2 碳化硅外延层双角度厚度测量结果（全谱 FFT）

测量参数	10°	15°
主频率 $S(\theta)$ (cm)	0.003936	0.003923
条纹周期 $\Delta\sigma$ (cm^{-1})	254.05	254.94
折射率 n	2.31	
厚度 $d(\theta)$ (μm)	8.53	8.53
相对差异 (%)	0.00	

表 3 碳化硅外延层双角度厚度测量结果（滑窗 FFT）

测量参数	10°	15°
厚度中位值 $d(\theta)$ (μm)	7.92	7.87
95% 置信区间 (μm)	[7.80, 8.04]	[7.77, 7.97]
相对 MAD (%)	0.76	0.66
厚度一致性（两角中位差异） (%)	0.68	

5.3 问题三的求解

硅外延层厚度的精确测量是半导体制造中的关键技术问题。传统的单角度两光束干涉模型在高精度测量场景下存在系统偏差，主要原因包括：

- (i) 多次反射产生的高阶干涉项被忽略；
- (ii) 单一约束条件导致参数估计的不唯一性；
- (iii) 仪器响应函数和材料色散效应的耦合误差。

5.3.1 多光束干涉模型建立

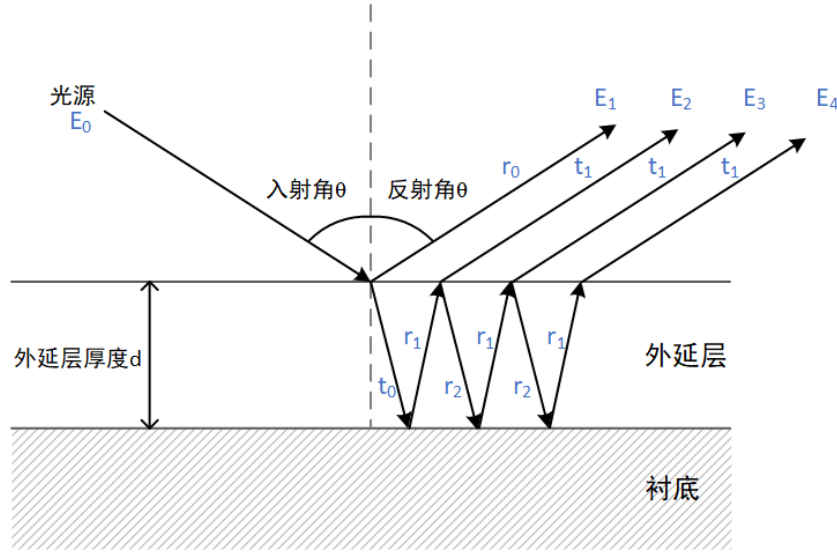


图 4 多光束干涉模型示意图

有一束光线射到硅晶圆片，光波在外延层界面和衬底界面产生多次反射和透射。设入射到外延层界面的复振幅为 E_0 ，第 i 束从外延层发射到空气中的反射光复振幅为 $E_i (i > 0)$ 。从空气透入到外延层界面的振幅透射系数为 t_0 ，从外延层透回空气的振幅透射系数为 t_1 。在外延层界面，从空气入射的振幅反射系数为 r_0 ，从衬底界面反射到外延层界面的振幅反射系数为 r_1 ，从外延层界面反射到衬底界面的振幅发射系数为 r_2 。光从空气入射到外延层，入射角为 θ ，折射角为 θ' ，空气和外延层界面的折射率为 $\tilde{n}_0, \tilde{n}_1 \in \mathbb{C}$ 。

5.3.2 多光束干涉的物理机制

各束光从外延层发射到空气中的反射光的复振幅为：

$$\begin{cases} E_1 = r_0 E_0, \\ E_2 = t_0 t_1 r_1 E_0 e^{-i\delta}, \\ E_3 = t_0 t_1 r_1 (r_1 r_2) E_0 e^{-i2\delta}, \\ E_4 = t_0 t_1 r_1 (r_1 r_2)^2 E_0 e^{-i3\delta}, \\ \vdots \\ E_n = t_0 t_1 r_1 (r_1 r_2)^{n-2} E_0 e^{-i(n-1)\delta}, \quad (n \geq 2) \end{cases}$$

将 E_1 - E_n 相干叠加，上述序列在 $|r_1 r_2| < 1$ 时收敛，因此得总复振幅^[3]

$$\begin{aligned} E_r &= r_0 E_0 + t_0 t_1 r_1 E_0 e^{-i\delta} \sum_{m=0}^{\infty} (r_1 r_2 e^{-i\delta})^m \\ &= r_0 E_0 + t_0 t_1 r_1 E_0 \frac{e^{-i\delta}}{1 - r_1 r_2 e^{-i\delta}}. \end{aligned}$$

Stokes 倒逆关系约束了无损、互易、各向同性界面的菲涅尔振幅系数。Stokes 倒逆关系由能量守恒与互易对称给出：

$$\begin{aligned} t_0 t_1 &= 1 - r_0^2, \\ r_2 &= -r_0 \end{aligned}$$

将 Stokes 倒逆关系带入到 E_r 求解得：

$$E_r = E_0 \frac{r_0 + r_1 e^{-i\delta}}{1 + r_0 r_1 e^{-i\delta}}$$

由此可得总振幅反射系数：

$$\begin{aligned} r_{total} &= \frac{E_r}{E_0} \\ &= \frac{r_0 + r_1 e^{-i\delta}}{1 + r_0 r_1 e^{-i\delta}} \end{aligned}$$

总反射率为：

$$\begin{aligned} R &= \left| \frac{E_r}{E_0} \right|^2 \\ &= \frac{r_0 + r_1 e^{-i\delta}}{1 + r_0 r_1 e^{-i\delta}} \times \frac{r_0 + r_1 e^{i\delta}}{1 + r_0 r_1 e^{i\delta}} \\ &= \frac{r_0^2 + r_1^2 + 2r_0 r_1 \cos \delta}{1 + r_0^2 r_1^2 + 2r_0 r_1 \cos \delta} \end{aligned}$$

令

$$x := r_0 r_1, \quad |x| < 1 \quad (\text{常见情形下成立}),$$

并注意到

$$1 + x^2 + 2x \cos \delta - (r_0^2 + r_1^2 + 2x \cos \delta) = (1 - r_0^2)(1 - r_1^2).$$

于是

$$R = 1 - \frac{(1 - r_0^2)(1 - r_1^2)}{1 + 2x \cos \delta + x^2}.$$

对分母用 Poisson 核的傅里叶展开 ($|x| < 1$):

$$\frac{1}{1 + 2x \cos \delta + x^2} = \frac{1}{1 - x^2} \left(1 + 2 \sum_{m=1}^{\infty} (-1)^m x^m \cos(m\delta) \right).$$

代回得到

$$R(\sigma) = R_0 + \sum_{m=1}^{\infty} A_m \cos(m\delta(\sigma)),$$

其中

$$R_0 = 1 - \frac{(1 - r_0^2)(1 - r_1^2)}{1 - x^2}, \quad A_m = \frac{2(1 - r_0^2)(1 - r_1^2)}{1 - x^2} (-1)^{m+1} x^m.$$

其中 δ 还是为两光束的相位差, $\delta = 4\pi n d \cos \theta' \sigma$ 。由此可以判断, 多光束并不会改变干涉相长的点的位置, 只会改变条纹的明暗程度。

5.3.3 多光束干涉的必要条件

界面的反射率 由空气、外延层、衬底三介质构成的单层膜, 出射到空气侧的光总复振幅是一个几何级数:

$$E_r = r_0 E_0 + t_0 t_1 r_1 E_0 e^{-i\delta} \sum_{m=0}^{\infty} (r_1 r_2 e^{-i\delta})^m$$

几何级数的“公比”是

$$q = r_0 r_1 e^{-i\delta}, \quad |q| = |r_0 r_1| = \sqrt{R_1 R_2},$$

其中 $R_1 = |r_0|^2$ (空气/外延界面能量反射率)、 $R_2 = |r_1|^2$ (外延/衬底界面能量反射率)。如果 $R_1 R_2 \ll 1$, 则从第二次往返开始 ($m \geq 1$), 强度项迅速降为首次内部束的一个极小分数, 几何级数几乎被首项“截断”, 多束效应退化为双束近似:

$$R(\delta) \approx R_1 + R_2 + 2\sqrt{R_1 R_2} \cos \delta.$$

让第二个在外延层内部往返光的强度至少占第一个外延层内部往返光的 α ， α 这里取 0.05

$$\frac{I_{m=2}}{I_{m=1}} \approx |q|^2 = R_1 R_2 \gtrsim \alpha \Rightarrow R_1 R_2 \gtrsim 0.05.$$

条纹尖锐度 条纹的“尖”度可以直观地描述是否存在多束光干涉，若存在多束光干涉则周期性图像会有尖端。半高全宽（FWHM）是在一条干涉峰（或共振峰）上，从峰值的一半高度两侧各落下去到曲线与这条“半高”水平线的两个交点之间的横向间距。自由谱距（FSR）是指相邻两条干涉极大之间的横向间距。在法布里-珀罗（Fabry-Perot）型的多光束干涉里，FWHM 描述的是每个干涉峰有多“尖”。多束光干涉的光强与 $|\tilde{E}_r|^2$ 正相关 $I \propto |\tilde{E}_r|^2$ ，因此多光束强度可形式化写成

$$I(\delta) = \frac{I_{\max}}{1 + F \sin^2(\delta/2)},$$

其中 δ 是相位差， F 是 finesse 系数， $F = \frac{4\sqrt{R_1 R_2}}{(1 - \sqrt{R_1 R_2})^2}$ ，由此得到 F 越大，峰越近。精细度 $\mathcal{F}$ 为自由谱距（FSR）与半高全宽（FWHM）的比 $\mathcal{F} = \frac{\text{FSR}}{\text{FWHM}}$ 。 $\mathcal{F} \gtrsim 1 \sim 2$ 时，尖峰明显，多束光线特征； $\mathcal{F} \ll 1$ 时近似正弦，双束光线特征。同样也是对 $R_1 R_2 \gtrsim \alpha \Rightarrow R_1 R_2 \gtrsim 0.05$ 。

5.3.4 计算精度影响

条纹形状的非线性化 多光束干涉导致干涉条纹从简单的正弦形状转变为尖峰-宽谷的非对称 Airy 形状。这种形状变化通过以下方式影响厚度精度：

峰位移效应：多光束干涉使干涉峰的位置相对于两光束情况发生微小偏移，偏移量为：

$$\Delta\sigma_{peak} = \frac{2|r_0 r_1|}{(1 - |r_0 r_1|^2)} \cdot \frac{\partial\phi}{\partial\sigma}$$

其中 ϕ 为相位差的相位部分。

峰形锐化效应：多光束干涉使半峰全宽 (FWHM) 显著减小：

$$FWHM_{multi} = FWHM_{two} \cdot \frac{1 - |r_0 r_1|}{2\sqrt{|r_0 r_1|}}$$

5.3.5 色散效应的耦合误差

在色散介质中，多光束干涉与色散效应耦合，产生额外的系统误差：

$$\Delta d_{dispersion} = \frac{2}{4} \cdot \frac{dn/d}{n^2} \cdot \mathcal{F} \cdot \Delta\sigma$$

5.3.6 精细度依赖的误差放大

精细度 $F = \frac{4\sqrt{R_1 R_2}}{(1 - \sqrt{R_1 R_2})^2}$ 的增加会放大多光束效应的影响。当 $\mathcal{F} > 3$ 时，忽略多光束效应导致的厚度误差为：

$$\frac{\Delta d}{d} \approx \frac{\mathcal{F}^2}{4^2} \cdot |r_0 r_1| \cdot \sin(\phi_{error})$$

5.3.7 具体算法

数据预处理 首先进行数据清洗，检查数据的完整性，去除 NaN 和无穷值检查，对波数单调性进行检查和排序并对反射率量纲统一化，将百分比转化为小数。随后裁切区域，去除低波数区域 ($< 400 \text{ cm}^{-1}$)，避免低频噪声。实施等距重采样将非等间距数据插值为等间距网格，计算波数间隔。并进行去趋势 + 去均值处理，3 次多项式拟合去除长期趋势，该预处理策略有效抑制了频谱泄漏现象，显著提升了频域特征提取的精度和可靠性。

FFT 检验多光束干涉模型 多光束干涉的情况如下：

$$R(\sigma) = R_0 + \sum_{m=1}^{\infty} A_m \cos(m \delta(\sigma)) \quad (1)$$

$$= R_0 + \sum_{m=1}^{\infty} A_m \cos(m 4\pi n d \cos \theta' \sigma) \quad (2)$$

$$= R_0 + \sum_{m=1}^{\infty} A_m \cos(2\pi S(\theta) \sigma) \quad (3)$$

$$= R_0 + A_1 \cos(2\pi S(\theta) \sigma) + A_2 \cos(4\pi S(\theta) \sigma) + A_3 \cos(6\pi S(\theta) \sigma) + \dots \quad (4)$$

FFT 谱中会出现多个谐波峰： $S(\theta), 2S(\theta), 3S(\theta), \dots$

其中硅取材料量级的折射率先验（硅 ~ 3.47 ）^[2]，对附件 3.csv 和附件 4.csv 做 FFT 后，可以观察到，除了一个明显的主波峰外，还有一个谐波峰，且主峰出现在 $S(\theta) = 0.25$ 处，谐波峰出现在 $2S(\theta) = 0.5$ 处。由此可以判断，该硅的测试结果出现了多光束干涉。

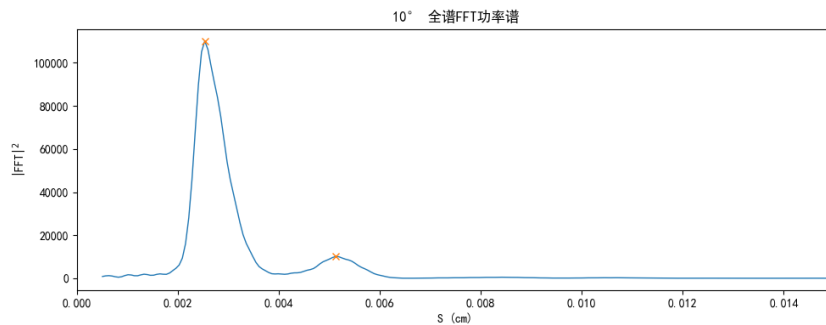


图 5 10° 入射角傅里叶

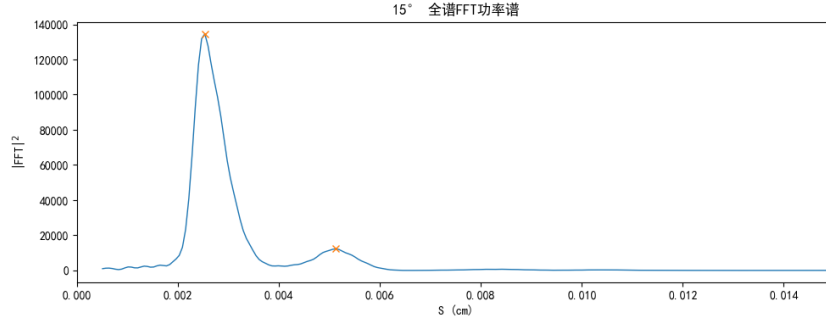


图 6 15° 入射角傅里叶

5.3.8 硅外延层厚度计算

在 FFT 谱中，由于多光束干涉的所有频率分量均源于相同的基本光程差 $\Delta = 2d\sqrt{n^2 - \sin^2 \theta}$ ，其中基频 $S(\theta) = 2d\sqrt{n^2 - \sin^2 \theta}$ 与几何厚度存在严格的一一对应关系，而 $2S(\theta)$ 、 $3S(\theta)$ 等谐波分量虽反映干涉机制的复杂性，但本质上为基频的整数倍，不提供额外的厚度约束信息，我们只关注主峰，求得 $S(\theta)$ 。根据 FFT 图求得主峰后，则可以代回 $S(\theta) = 2d\sqrt{n^2 - \sin^2 \theta}$ ，求得

$$d = \frac{S(\theta)}{2\sqrt{n^2 - \sin^2 \theta}}$$

5.3.9 结果及一致性评估

表 4 硅外延层双角度厚度测量结果

测量参数	10°	15°
主频率 $S(\theta)$ (cm)	0.002536	0.002519
条纹周期 $\Delta\sigma$ (cm ⁻¹)	394.34	397.04
厚度 $d(\theta)$ (μm) ¹	3.66	3.64
相对差异 (%)	0.53	

主峰在 0.002532 cm，二次峰：0.005127 cm，二次谐波比值 ≈ 2.02 ，非常接近理论值 2.0 这是典型的多光束干涉特征，多束光效应显著。检测到 10° 入射角的主峰在 $S(10^\circ) \approx 0.002536 \text{ cm}$ ，15° 入射角的主峰在 $S(15^\circ) \approx 0.002519 \text{ cm}$ ，则

$$d(10^\circ) = \frac{S(10^\circ)}{2\sqrt{3.47^2 - \sin^2(10^\circ)}} = 3.66 \mu\text{m}$$

$$d(15^\circ) = \frac{S(10^\circ)}{2\sqrt{3.47^2 - \sin^2(15^\circ)}} = 3.64\mu m$$

两角一致性:

$$\frac{|d(10^\circ) - d(15^\circ)|}{\frac{d(10^\circ) + d(15^\circ)}{2}} = 0.53\%$$

因此两角一致性很高，可靠性很高。

5.3.10 多束光对碳化硅外延层厚度测量的影响

研究是否多束光干涉出现在碳化硅晶圆片的测试结果中可以运用谐波检测法。我们对附件 1.csv 和附件 2.csv 做 FFT 图谱分析。首先得到主峰出现在 $S(\theta)$ 处，接着我们在 FFT 图谱中寻找 $2S(\theta), 3S(\theta)...$ 等位置的峰值，设定阈值，如果超过阈值则判定为多光束干涉。经实验得到下图结果：FFT 后得到的 $\sigma - S$ 图，观察到只在 $S(\theta) \approx 0.004\text{cm}$ 处看到了一条明显的亮条纹，而在 $2S(\theta), 3S(\theta)...$ 等位置没有明显的两条纹路。

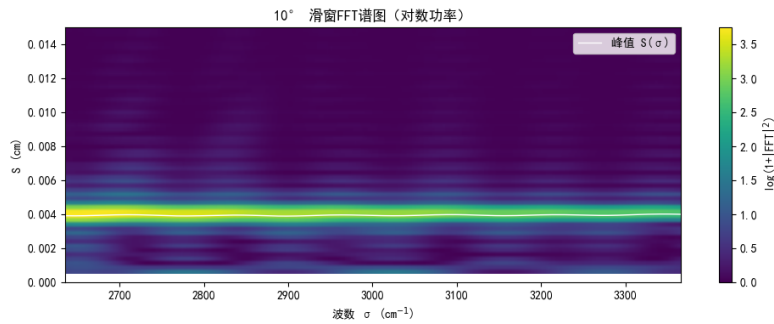


图 7 10° 入射角窗口傅里叶

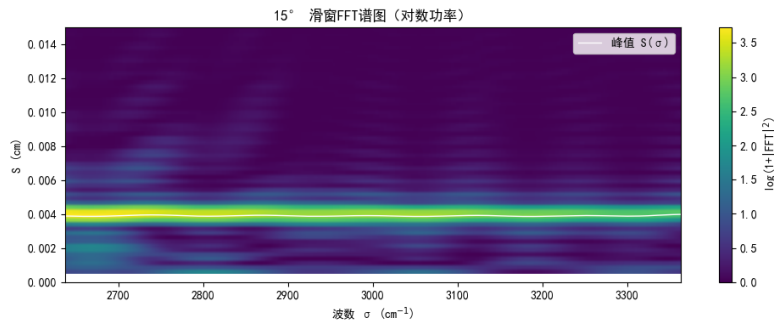


图 8 15° 入射角窗口傅里叶

因此认为多束光干涉并没有出现在碳化硅晶圆片的测试中，如果多束光干涉出现在碳化硅晶圆片的测试中，依然可以用 FFT，求得主峰所在的位置 $S(\theta)$ ，再代入公式 $d = \frac{S(\theta)}{2\sqrt{n^2 - \sin^2 \theta}}$ ，仍然可以求出外延层的厚度，且精度不受太大影响。

六、模型评价

6.1 模型优点

6.1.1 技术创新性优势

本研究提出的基于全谱 FFT 与滑窗 FFT 协同的厚度测量方法具有显著的技术创新性。全谱 FFT-滑窗 FFT 双重分析策略实现了从全局特征提取到局域机制识别的多尺度解析，这一设计理念在薄膜厚度测量领域具有开创性意义。与传统的单一 FFT 方法相比，该策略能够同时获得厚度信息和干涉机制特征，为复杂光学系统的精确表征提供了新的技术路径。

6.1.2 计算效率与自动化优势

模型采用标量近似处理，显著简化了 Fresnel 反射系数的计算复杂度，使得算法在保持合理精度的同时大幅提升了计算效率。FFT 变换的 $O(N \log N)$ 时间复杂度相比传统的逐点拟合方法具有显著的速度优势，特别适用于大数据量的在线测量需求。此外，主峰自动识别算法减少了人工干预，提高了测量过程的自动化程度和可重现性。

6.2 模型缺点

6.2.1 基本假设的理论局限

模型采用的标量近似忽略了偏振效应，这在大角度入射或具有各向异性的材料系统中可能引入显著误差。对于入射角超过 30° 或双折射材料，s 偏振与 p 偏振的差异可能导致厚度测量精度的明显下降。此外，单层模型假设限制了其在多层异质结构中的直接应用，需要进一步的理论扩展。

6.2.2 频域分析的固有限制

FFT 方法要求信号具有良好的周期性特征，对于厚度过薄 ($<0.5 \mu\text{m}$) 或过厚 ($>100 \mu\text{m}$) 的样品，干涉条纹的可观测性可能受限。当干涉条纹周期接近仪器分辨率极限时，频域峰的识别精度将显著降低。同时，表面粗糙度、界面扩散等因素引起的谱线展宽可能影响主峰位置的准确确定。

6.2.3 材料参数的先验依赖

模型的初值设定依赖于材料折射率的先验知识（如 Si 的 $n_0=3.4$ ），这在处理未知组分或新型材料时可能成为限制因素。当实际材料参数与先验值偏差较大时，可能导致拟合过程收敛到局部最优解，影响最终结果的准确性。

参考文献

- [1] ChatGPT, GPT-5, OpenAI, 2025-09-06.
- [2] Polyanskiy M. N. Refractive index of SiC (Silicon carbide) —4H-SiC, ordinary (Wang et al. 2013)[EB/OL]. RefractiveIndex.INFO, 2025-09-07[引用日期]. :<https://refractiveindex.info/?book=SiC&page=Wang-4H-o&shelf=main>.
- [3] Ismail N., Kores C. C., Geskus D., Pollnau M. The Fabry-Pérot resonator: Spectral line shapes, generic and related Airy distributions, linewidths, finesses, and performance at low or frequency-dependent reflectivity[J]. *Optics Express*, 2016, 24(15): 16366–16389. <https://opg.optica.org/oe/fulltext.cfm?uri=oe-24-15-16366&id=346183>.
- [4] <https://zhuanlan.zhihu.com/p/1947685435766710483>.
- [5] Polyanskiy M. N. Refractive index of Si (Silicon) —Shkondin et al. 2012[EB/OL]. RefractiveIndex.INFO, 2025-09-07[引用日期]. <https://refractiveindex.info/?shelf=main&book=Si&page=Shkondin>.

附录 A

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from scipy.signal import savgol_filter
from scipy.interpolate import interp1d

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

sp10 = Path("附件1.xlsx") # 一次10°反射透射
sp15 = Path("附件2.xlsx") # 一次15°反射透射

# 读取文件函数
def read(path: Path) -> pd.DataFrame:
    df = pd.read_excel(path)
    cols = ["波数 (cm-1)", "反射率 (%)"]
    df.columns = cols
    out = df[[cols[0], cols[1]]].copy()
    out.columns = ["wn", "r"]
    out = out.apply(pd.to_numeric, errors="coerce").dropna()
    out = out.sort_values("wn").reset_index(drop=True)
    return out

# 读取数据并画出原始曲线
sp10_df = read(sp10)
sp15_df = read(sp15)

plt.figure(figsize=(10,4))
plt.plot(sp10_df["wn"], sp10_df["r"], label="10°原始")
plt.plot(sp15_df["wn"], sp15_df["r"], label="15°原始")
plt.xlabel("波数 (cm-1)"); plt.ylabel("反射率 (%)"); plt.title("原始反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 裁切, 及进行傅里叶变换前的预处理 [1] ChatGPT, GPT-5, OpenAI, 2025-09-06
# 取出原始数组 (方便一起裁剪)
wn10_all = sp10_df["wn"].to_numpy()
r10_all = sp10_df["r"].to_numpy()
wn15_all = sp15_df["wn"].to_numpy()
r15_all = sp15_df["r"].to_numpy()
CUTOFF=2000.0
def cut(wn, y_sg, y_raw=None, cutoff=2000.0):
    wn = wn.astype(float)
    mask = (wn >= cutoff)
```

```

    wn_cut = wn[mask]
    y_sg_cut = y_sg[mask]
    y_raw_cut = y_raw[mask] if y_raw is not None else None
    return wn_cut, y_sg_cut, y_raw_cut
wn10_cut, sp10_cut, r10_cut = cut(wn10_all, sp10_df["r"], r10_all)
wn15_cut, sp15_cut, r15_cut = cut(wn15_all, sp15_df["r"], r15_all)

plt.figure(figsize=(10,4))
plt.plot(wn15_cut, sp15_cut, label="15° SG")
plt.plot(wn10_cut, sp10_cut, label="10° SG")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率 (%)"); plt.title("裁剪后反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 等间距重采样
def rewrite(wn, y):
    wn = np.asarray(wn, float)
    y = np.asarray(y, float)
    newwn = np.linspace(wn.min(), wn.max(), len(wn))
    newy = np.interp(newwn, wn, y)
    dsigma = newwn[1] - newwn[0]
    return newwn, newy, dsigma

# 去趋势
def detrend(x, y):
    X = np.vstack([x**k for k in range(4)]).T
    coef, *_ = np.linalg.lstsq(X, y, rcond=None)
    return y - X @ coef

def prep(wn, y):
    y_d = detrend(wn, y)
    y_d = y_d - np.mean(y_d)
    return y_d

wn10_u, sp10_u, delta10 = rewrite(wn10_cut, sp10_cut)
sp10_p = prep(wn10_u, sp10_u)
wn15_u, sp15_u, delta15 = rewrite(wn15_cut, sp15_cut)
sp15_p = prep(wn15_u, sp15_u)

# 输出图像
plt.figure(figsize=(10,4))
plt.plot(wn10_u, sp10_p, label="10°预处理")
plt.plot(wn15_u, sp15_p, label="15°预处理")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率 (%)"); plt.title("预处理后反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 更好的峰值
def betterpeak(F, P, k):
    n = len(P)

```

```

    if k <= 0 or k >= n - 1:
        return float(F[k])
    y1, y2, y3 = P[k-1], P[k], P[k+1]
    denom = y1 - 2.0 * y2 + y3
    if abs(denom) < 1e-15:
        return float(F[k])
    delta = 0.5 * (y1 - y3) / denom
    df = F[1] - F[0]
    f = F[k] + delta * df
    return f

# 全谱FFT求全局S(), FFT模板由人工智能生成 [1] ChatGPT, GPT-5, OpenAI, 2025-09-07
def globalfft(y, delta, cut=0):
    n = len(y)
    N = int(2**int(np.log2(n)+2.5))
    Y = np.fft.fft(y, n=N)
    F = np.fft.fftfreq(N, d=delta)
    P = np.abs(Y)**2
    m = F >= cut
    F, P = F[m], P[m]
    k = int(np.argmax(P))
    S = betterpeak(F, P, k)
    return S

# 滑窗 FFT求局部S()
def slidefft(wn, y, delta, T=5, forwardT=0.01, cut=5e-4):
    S0= globalfft(y, delta, cut)
    dsigma = 1.0 / S0
    winp = int(T * dsigma / delta)
    sigmas, Ss = [], []
    start = 0
    end = len(wn) - winp + 1
    step = int(forwardT * dsigma / delta)
    for s in range(start, end, step):
        seg = y[s:s+winp]
        N = int(2**int(np.log2(winp)+2.5))
        Y = np.fft.fft(seg, n=N)
        F = np.fft.fftfreq(N, d=delta)
        P = np.abs(Y)**2
        m = F >= cut
        F, P = F[m], P[m]
        k = int(np.argmax(P))
        S= betterpeak(F, P, k)
        Ss.append(S)
        sigmas.append(0.5*(wn[s] + wn[s+winp-1]))
    return np.array(sigmas), np.array(Ss)

```

```

def check(x):
    x = np.asarray(x, float)
    x = x[np.isfinite(x)]
    if len(x) == 0:
        return np.nan, np.nan, np.nan
    med = np.median(x)
    mad = np.median(np.abs(x - med))
    sigma_hat = 1.4826 * mad
    rel_mad_pct = (sigma_hat / max(abs(med), 1e-12)) * 100.0
    return float(med), float(med - 1.96*sigma_hat), float(med + 1.96*sigma_hat),
        float(rel_mad_pct)

def answer(s10,s15):
    sin10, sin15 = np.sin(np.deg2rad(10.0)), np.sin(np.deg2rad(15.0))
    n = 2.5
    n2=2.5*2.5
    d10 = s10 / (2 * np.sqrt(n2 - sin10 ** 2))
    d15 = s15 / (2 * np.sqrt(n2 - sin15 ** 2))
    return d10, d15

# 全谱 FFT: 给 S 的全局初值
s10 = globalfft(sp10_p, delta10)
s15 = globalfft(sp15_p, delta15)
d10a, d15a = answer(s10,s15)
print(f"[全谱FFT]:")
print(f"S(10°)   {s10.real:.6f} cm, Δ   {1/s10.real:.2f} cm-1")
print(f"S(15°)   {s15.real:.6f} cm, Δ   {1/s15.real:.2f} cm-1")
print(f"d(10°)   {d10a*10000:.2f} um, d(15°)   {d15a*10000:.2f} um ")

# 滑窗 FFT: 得到 S() 的局部估计

sigmas10, Ss10 = slidefft(wn10_u, sp10_p, delta10)
sigmas15, Ss15 = slidefft(wn15_u, sp15_p, delta15)
left = max(np.nanmin(sigmas10), np.nanmin(sigmas15))
right = min(np.nanmax(sigmas10), np.nanmax(sigmas15))

# 作图看 S() 的稳定性
plt.figure(figsize=(10,4))
plt.plot(sigmas10, Ss10, '-o', ms=3, label="S10() • 滑窗FFT")
plt.plot(sigmas15, Ss15, '-o', ms=3, label="S15() • 滑窗FFT")
plt.xlabel("波数 (cm-1)"); plt.ylabel("S() (cm)")
plt.title("局部条纹频率 S() (滑窗FFT)")
plt.legend(); plt.tight_layout(); plt.show()

d10_sigma=Ss10/(2*np.sqrt((0.00005* sigmas10+2.34)**2-np.sin(np.deg2rad(10))**2))
d15_sigma=Ss15/(2*np.sqrt((0.00005* sigmas15+2.34)**2-np.sin(np.deg2rad(15))**2))
m10 = np.isfinite(d10_sigma) & (sigmas10 >= left) & (sigmas10 <= right)

```

```

m15 = np.isfinite(d15_sigma) & (sigmas15 >= left) & (sigmas15 <= right)
# 置信检验 (稳健统计)
d10_med, d10_lo, d10_hi, rel_d10_mad_pct = check(d10_sigma)
d15_med, d15_lo, d15_hi, rel_d15_mad_pct = check(d15_sigma)
d_med_ref = np.median([d10_med, d15_med])
diff_pct = abs(d10_med - d15_med) / max(abs(d_med_ref), 1e-12) * 100.0

f15 = interp1d(sigmas15[m15], d15_sigma[m15], kind='linear', bounds_error=False,
               fill_value=np.nan)
x_common = sigmas10[m10]
y10_common = d10_sigma[m10] * 1e4
y15_on_10 = f15(x_common) * 1e4

plt.figure(figsize=(10,4))
plt.plot(x_common, y10_common, '-o', ms=3, label="d() • 10°回代")
plt.plot(x_common, y15_on_10, '-o', ms=3, label="d() • 15°回代 (插值到10°网格)", alpha=0.8)
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("厚度 d (m)")
plt.title("厚度 d() (同一 网格对齐)")
plt.legend(); plt.tight_layout(); plt.show()

print("[滑动FFT]:")
print(f"d()10°: 中位 {d10_med*10000:.2f} m, 约95%区间 [{d10_lo*10000:.2f}, {d10_hi*10000:.2f}] m, 相对MAD {rel_d10_mad_pct:.2f}%")
print(f"d()15°: 中位 {d15_med*10000:.2f} m, 约95%区间 [{d15_lo*10000:.2f}, {d15_hi*10000:.2f}] m, 相对MAD {rel_d15_mad_pct:.2f}%")
print(f"厚度一致性 (两角中位差异) {diff_pct:.2f}%")

```

附录 B

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from scipy.interpolate import interp1d
from scipy.signal import find_peaks

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

sp10 = Path("附件3.xlsx") # 一次10°反射透射
sp15 = Path("附件4.xlsx") # 一次15°反射透射
n=3.47 # 硅折射率（近似常数）

# 读取文件函数
def read(path: Path) -> pd.DataFrame:
    df = pd.read_excel(path)
    cols = ["波数 (cm-1)", "反射率 (%)"]
    df.columns = cols
    out = df[[cols[0], cols[1]]].copy()
    out.columns = ["wn", "rx"]
    out = out.apply(pd.to_numeric, errors="coerce").dropna()
    out = out.sort_values("wn").reset_index(drop=True)
    return out

# 读取数据并画出原始曲线
sp10_df = read(sp10)
sp15_df = read(sp15)
sp10_df["r"] = sp10_df["rx"] / 100.0
sp15_df["r"] = sp15_df["rx"] / 100.0
plt.figure(figsize=(10,4))
plt.plot(sp10_df["wn"], sp10_df["r"], label="10°原始")
plt.plot(sp15_df["wn"], sp15_df["r"], label="15°原始")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率"); plt.title("原始反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 裁切
wn10_all = sp10_df["wn"].to_numpy()
r10_all = sp10_df["r"].to_numpy()
wn15_all = sp15_df["wn"].to_numpy()
r15_all = sp15_df["r"].to_numpy()
CUTOFF=400.0
def cut(wn, y_sg, y_raw=None, cutoff=400.0):
    wn = wn.astype(float)
```



```

mask = (wn >= cutoff)
wn_cut = wn[mask]
y_sg_cut = y_sg[mask]
y_raw_cut = y_raw[mask] if y_raw is not None else None
return wn_cut, y_sg_cut, y_raw_cut

wn10_cut, sp10_cut, r10_cut = cut(wn10_all, sp10_df["r"], r10_all)
wn15_cut, sp15_cut, r15_cut = cut(wn15_all, sp15_df["r"], r15_all)

plt.figure(figsize=(10,4))
plt.plot(wn15_cut, sp15_cut, label="15° SG")
plt.plot(wn10_cut, sp10_cut, label="10° SG")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率"); plt.title("裁剪后反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 等间距重采样
def rewrite(wn, y):
    wn = np.asarray(wn, float)
    y = np.asarray(y, float)
    newwn = np.linspace(wn.min(), wn.max(), len(wn))
    newy = np.interp(newwn, wn, y)
    dsigma = newwn[1] - newwn[0]
    return newwn, newy, dsigma

# 去趋势
def detrend(x, y):
    X = np.vstack([x**k for k in range(4)]).T
    coef, *_ = np.linalg.lstsq(X, y, rcond=None)
    return y - X @ coef

def prep(wn, y):
    y_d = detrend(wn, y)
    y_d = y_d - np.mean(y_d)
    return y_d

# 处理 10° 曲线
wn10_u, sp10_u, delta10 = rewrite(wn10_cut, sp10_cut)
sp10_p = prep(wn10_u, sp10_u)
# 处理 15° 曲线
wn15_u, sp15_u, delta15 = rewrite(wn15_cut, sp15_cut)
sp15_p = prep(wn15_u, sp15_u)
# 输出图像
plt.figure(figsize=(10,4))
plt.plot(wn10_u, sp10_p, label="10°预处理")
plt.plot(wn15_u, sp15_p, label="15°预处理")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率"); plt.title("预处理后反射谱")
plt.legend(); plt.tight_layout(); plt.show()

```

```

# 更好的峰值
def betterpeak(F, P, k):
    n = len(P)
    if k <= 0 or k >= n - 1:
        return float(F[k])
    y1, y2, y3 = P[k-1], P[k], P[k+1]
    denom = y1 - 2.0 * y2 + y3
    if abs(denom) < 1e-15:
        return float(F[k])
    delta = 0.5 * (y1 - y3) / denom
    df = F[1] - F[0]
    f = F[k] + delta * df
    return f

# 全谱FFT求全局S()
def globalfft(y, delta, cut=0, ret=False):
    n = len(y)
    N = int(2**int(np.log2(n)+2.5))
    Y = np.fft.fft(y, n=N)
    F = np.fft.fftfreq(N, d=delta)
    P = np.abs(Y)**2
    m = F >= cut
    F, P = F[m], P[m]
    k = int(np.argmax(P))
    S = float(betterpeak(F, P, k))
    if ret:
        return S, F, P, k
    return S

# 滑窗 FFT求局部S()
def slidefft(wn, y, delta, T=7, forwardT=0.01, cut=5e-4, return_spec=False):
    S0, _, _, _ = globalfft(y, delta, cut, ret=True)
    dsigma = 1.0 / S0
    winp = int(T * dsigma / delta)
    if winp < 3:
        if return_spec:
            return np.array([]), np.array([]), np.array([]), np.empty((0, 0))
        else:
            return np.array([]), np.array([])

    step = max(1, int(forwardT * dsigma / delta))
    start = 0
    end = len(wn) - winp + 1

    sigmas, Ss = [], []
    P_rows = []

```

```

F_ref = None

for s in range(start, end, step):
    seg = y[s:s+winp]
    N = int(2**int(np.log2(winp)+2.5))
    Y = np.fft.fft(seg, n=N)
    F = np.fft.fftfreq(N, d=delta)
    P = np.abs(Y)**2
    m = F >= cut
    F, P = F[m], P[m]
    if F_ref is None:
        F_ref = F
    else:
        if len(F) != len(F_ref) or np.max(np.abs(F - F_ref)) > 1e-12:
            P = np.interp(F_ref, F, P, left=0.0, right=0.0)
            F = F_ref
        k = int(np.argmax(P))
        S = betterpeak(F, P, k)
        Ss.append(S)
        sigmas.append(0.5 * (wn[s] + wn[s+winp-1]))
    if return_spec:
        P_rows.append(P)
sigmas = np.array(sigmas)
Ss = np.array(Ss)
if return_spec:
    P_mat = np.vstack(P_rows) if P_rows else np.empty((0, 0))
    return sigmas, Ss, F_ref, P_mat
return sigmas, Ss

def answer(s10,s15):
    sin10, sin15 = np.sin(np.deg2rad(10.0)), np.sin(np.deg2rad(15.0))
    n2 = 3.47**2
    n = 3.47
    d10 = s10 / (2 * np.sqrt(n2 - sin10 ** 2))
    d15 = s15 / (2 * np.sqrt(n2 - sin15 ** 2))
    return n, d10, d15

def plot_global_spectrum(F, P, title=""):
    plt.figure(figsize=(10, 4))
    plt.plot(F, P, lw=1)
    y = find_peaks(P, height=8000)
    plt.plot(F[y[0]], y[1]['peak_heights'], 'x')
    plt.xlim(0, 0.015)
    plt.xlabel("S (cm)")
    plt.ylabel("|FFT|2")
    plt.title(title)
    plt.tight_layout(); plt.show()

```

```

print(f"检测到的峰值频率S: {F[y[0]][0]:.6f}(),
      {F[y[0]][1]:.6f}({F[y[0]][1]/F[y[0]][0]:.2f})")

def plot_spectrogram(sigmas, F, P_mat, Ss=None, title=""):
    if P_mat.size == 0:
        return
    plt.figure(figsize=(10, 4))
    extent = [sigmas.min(), sigmas.max(), F.min(), F.max()]
    plt.imshow(np.log1p(P_mat).T, extent=extent, origin="lower", aspect="auto")
    plt.ylim(0, 0.015)
    if Ss is not None and len(Ss) == len(sigmas):
        plt.plot(sigmas, Ss, 'w-', lw=1, label="峰值 S()")
        plt.legend(loc="upper right")
    plt.xlabel("波数 (cm-1)")
    plt.ylabel("S (cm)")
    plt.title(title + " (对数功率)")
    plt.colorbar(label="log(1+|FFT|2)")
    plt.tight_layout(); plt.show()

print(f"代入已知数据: 硅晶体在400到4000波数下n = 3.47 (近似常数)")
# 全谱 FFT: 给 S 的全局初值
s10, F10, P10, _ = globalfft(sp10_p, delta10, cut=5e-4, ret=True)
s15, F15, P15, _ = globalfft(sp15_p, delta15, cut=5e-4, ret=True)

# 画全局频谱 (横轴是 S, 纵轴是 |FFT|2)
plot_global_spectrum(F10, P10, "10° 全谱FFT功率谱")
plot_global_spectrum(F15, P15, "15° 全谱FFT功率谱")

n, d10a, d15a = answer(s10, s15)
print(f"[全谱FFT]:")
print(f"S(10°) {s10.real:.6f} cm, Δ {1/s10.real:.2f} cm-1")
print(f"S(15°) {s15.real:.6f} cm, Δ {1/s15.real:.2f} cm-1")
print(f"d(10°) {d10a*10000:.2f} um, d(15°) {d15a*10000:.2f} um ")
diff=abs(d10a - d15a)
diff_pct = diff / ((d10a + d15a)/2) * 100
print(f"厚度一致性 (两角中位差异) {diff_pct:.2f}%")

```

附录 C

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from scipy.interpolate import interp1d
from scipy.signal import find_peaks

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

sp10 = Path("附件1.xlsx") # 一次10°反射透射
sp15 = Path("附件2.xlsx") # 一次15°反射透射

# 读取文件函数
def read(path: Path) -> pd.DataFrame:
    df = pd.read_excel(path)
    cols = ["波数 (cm-1)", "反射率 (%)"]
    df.columns = cols
    out = df[[cols[0], cols[1]]].copy()
    out.columns = ["wn", "rx"]
    out = out.apply(pd.to_numeric, errors="coerce").dropna()
    out = out.sort_values("wn").reset_index(drop=True)
    return out

# 读取数据并画出原始曲线
sp10_df = read(sp10)
sp15_df = read(sp15)
sp10_df["r"] = sp10_df["rx"] / 100.0
sp15_df["r"] = sp15_df["rx"] / 100.0
plt.figure(figsize=(10,4))
plt.plot(sp10_df["wn"], sp10_df["r"], label="10°原始")
plt.plot(sp15_df["wn"], sp15_df["r"], label="15°原始")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率"); plt.title("原始反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 裁切
wn10_all = sp10_df["wn"].to_numpy()
r10_all = sp10_df["r"].to_numpy()
wn15_all = sp15_df["wn"].to_numpy()
r15_all = sp15_df["r"].to_numpy()
CUTOFF=2000.0
def cut(wn, y_sg, y_raw=None, cutoff=2000.0):
    wn = wn.astype(float)
    mask = (wn >= cutoff)
```

```

    wn_cut = wn[mask]
    y_sg_cut = y_sg[mask]
    y_raw_cut = y_raw[mask] if y_raw is not None else None
    return wn_cut, y_sg_cut, y_raw_cut

wn10_cut, sp10_cut, r10_cut = cut(wn10_all, sp10_df["r"], r10_all)
wn15_cut, sp15_cut, r15_cut = cut(wn15_all, sp15_df["r"], r15_all)

plt.figure(figsize=(10,4))
plt.plot(wn15_cut, sp15_cut, label="15° SG")
plt.plot(wn10_cut, sp10_cut, label="10° SG")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率"); plt.title("裁剪后反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 等间距重采样
def rewrite(wn, y):
    wn = np.asarray(wn, float)
    y = np.asarray(y, float)
    newwn = np.linspace(wn.min(), wn.max(), len(wn))
    newy = np.interp(newwn, wn, y)
    dsigma = newwn[1] - newwn[0]
    return newwn, newy, dsigma

# 去趋势
def detrend(x, y):
    X = np.vstack([x**k for k in range(4)]).T
    coef, *_ = np.linalg.lstsq(X, y, rcond=None)
    return y - X @ coef

def prep(wn, y):
    y_d = detrend(wn, y)
    y_d = y_d - np.mean(y_d)
    return y_d

# 处理曲线
wn10_u, sp10_u, delta10 = rewrite(wn10_cut, sp10_cut)
sp10_p = prep(wn10_u, sp10_u)
wn15_u, sp15_u, delta15 = rewrite(wn15_cut, sp15_cut)
sp15_p = prep(wn15_u, sp15_u)

# 输出图像
plt.figure(figsize=(10,4))
plt.plot(wn10_u, sp10_p, label="10°预处理")
plt.plot(wn15_u, sp15_p, label="15°预处理")
plt.xlabel("波数 (cm$^{-1}$)"); plt.ylabel("反射率"); plt.title("预处理后反射谱")
plt.legend(); plt.tight_layout(); plt.show()

# 更好的峰值

```

```

def betterpeak(F, P, k):
    n = len(P)
    if k <= 0 or k >= n - 1:
        return float(F[k])
    y1, y2, y3 = P[k-1], P[k], P[k+1]
    denom = y1 - 2.0 * y2 + y3
    if abs(denom) < 1e-15:
        return float(F[k])
    delta = 0.5 * (y1 - y3) / denom
    df = F[1] - F[0]
    f = F[k] + delta * df
    return f

# 全谱FFT求全局S()
def globalfft(y, delta, cut=0, ret=False):
    n = len(y)
    N = int(2**int(np.log2(n)+2.5))
    Y = np.fft.fft(y, n=N)
    F = np.fft.fftfreq(N, d=delta)
    P = np.abs(Y)**2
    m = F >= cut
    F, P = F[m], P[m]
    k = int(np.argmax(P))
    S = float(betterpeak(F, P, k))
    if ret:
        return S, F, P, k
    return S

# 滑窗 FFT求局部S()
def slidefft(wn, y, delta, T=5, forwardT=0.01, cut=5e-4, return_spec=False):
    S0, _, _, _ = globalfft(y, delta, cut, ret=True)
    dsigma = 1.0 / S0
    winp = int(T * dsigma / delta)
    if winp < 3:
        if return_spec:
            return np.array([]), np.array([]), np.array([]), np.empty((0, 0))
        else:
            return np.array([]), np.array([])

    step = max(1, int(forwardT * dsigma / delta))
    start = 0
    end = len(wn) - winp + 1

    sigmas, Ss = [], []
    P_rows = []
    F_ref = None

```

```

for s in range(start, end, step):
    seg = y[s:s+winp]
    N = int(2**int(np.log2(winp)+2.5))
    Y = np.fft.fft(seg, n=N)
    F = np.fft.fftfreq(N, d=delta)
    P = np.abs(Y)**2
    m = F >= cut
    F, P = F[m], P[m]
    if F_ref is None:
        F_ref = F
    else:
        if len(F) != len(F_ref) or np.max(np.abs(F - F_ref)) > 1e-12:
            P = np.interp(F_ref, F, P, left=0.0, right=0.0)
            F = F_ref
    k = int(np.argmax(P))
    S = betterpeak(F, P, k)
    Ss.append(S)
    sigmas.append(0.5 * (wn[s] + wn[s+winp-1]))
    if return_spec:
        P_rows.append(P)
sigmas = np.array(sigmas)
Ss = np.array(Ss)
if return_spec:
    P_mat = np.vstack(P_rows) if P_rows else np.empty((0, 0))
    return sigmas, Ss, F_ref, P_mat
return sigmas, Ss

def check(x):
    x = np.asarray(x, float)
    x = x[np.isfinite(x)]
    if len(x) == 0:
        return np.nan, np.nan, np.nan
    med = np.median(x)
    mad = np.median(np.abs(x - med))
    sigma_hat = 1.4826 * mad
    rel_mad_pct = (sigma_hat / max(abs(med), 1e-12)) * 100.0
    return float(med), float(med - 1.96*sigma_hat), float(med + 1.96*sigma_hat),
        float(rel_mad_pct)

def answer(s10,s15):
    sin10, sin15 = np.sin(np.deg2rad(10.0)), np.sin(np.deg2rad(15.0))
    n = 2.5
    n2=2.5*2.5
    d10 = s10 / (2 * np.sqrt(n2 - sin10 ** 2))
    d15 = s15 / (2 * np.sqrt(n2 - sin15 ** 2))
    return d10, d15

```



```

def plot_global_spectrum(F, P, title=""):
    plt.figure(figsize=(10, 4))
    plt.plot(F, P, lw=1)
    y = find_peaks(P, height=20)
    plt.plot(y[0], P[y[0]], 'ro', label="峰值")
    plt.xlim(0, 0.015)
    plt.xlabel("S (cm)")
    plt.ylabel("|FFT| $^2$ ")
    plt.title(title)
    plt.tight_layout(); plt.show()

def plot_spectrogram(sigmas, F, P_mat, Ss=None, title=""):
    if P_mat.size == 0:
        return
    plt.figure(figsize=(10, 4))
    extent = [sigmas.min(), sigmas.max(), F.min(), F.max()]
    plt.imshow(np.log1p(P_mat).T, extent=extent, origin="lower", aspect="auto")
    plt.ylim(0, 0.015)
    if Ss is not None and len(Ss) == len(sigmas):
        plt.plot(sigmas, Ss, 'w-', lw=1, label="峰值 S()")
        plt.legend(loc="upper right")
    plt.xlabel("波数 (cm $^{-1}$ )")
    plt.ylabel("S (cm)")
    plt.title(title + " (对数功率)")
    plt.colorbar(label="log(1+|FFT| $^2$ )")
    plt.tight_layout(); plt.show()

# 全谱 FFT: 给 S 的全局初值
s10, F10, P10, _ = globalfft(sp10_p, delta10, cut=5e-4, ret=True)
s15, F15, P15, _ = globalfft(sp15_p, delta15, cut=5e-4, ret=True)

# 画全局频谱 (横轴是 S, 纵轴是 |FFT| $^2$ )
plot_global_spectrum(F10, P10, "10° 全谱FFT功率谱")
plot_global_spectrum(F15, P15, "15° 全谱FFT功率谱")

d10a, d15a = answer(s10, s15)
print(f"[全谱FFT]:")
print(f"S(10°) {s10.real:.6f} cm, Δ {1/s10.real:.2f} cm $^{-1}$ ")
print(f"S(15°) {s15.real:.6f} cm, Δ {1/s15.real:.2f} cm $^{-1}$ ")
print("在S=2, 3...附近均未发现明显峰值, 认为没有发生多光束干涉")

# 滑窗 FFT: 得到 S() 的局部估计

sigmas10, Ss10, Fw10, Pm10 = slidefft(wn10_u, sp10_p, delta10, return_spec=True)
sigmas15, Ss15, Fw15, Pm15 = slidefft(wn15_u, sp15_p, delta15, return_spec=True)

# 画 “—S” 谱图 (每个 的窗口给出一条功率谱, 颜色表示能量)

```

```

plot_spectrogram(sigmas10, Fw10, Pm10, Ss10, "10° 滑窗FFT谱图")
plot_spectrogram(sigmas15, Fw15, Pm15, Ss15, "15° 滑窗FFT谱图")

left = max(np.nanmin(sigmas10), np.nanmin(sigmas15))
right = min(np.nanmax(sigmas10), np.nanmax(sigmas15))

d10_sigma=Ss10/(2*np.sqrt((0.00005* sigmas10+2.34)**2-np.sin(np.deg2rad(10))**2))
d15_sigma=Ss15/(2*np.sqrt((0.00005* sigmas15+2.34)**2-np.sin(np.deg2rad(15))**2))
m10 = np.isfinite(d10_sigma) & (sigmas10 >= left) & (sigmas10 <= right)
m15 = np.isfinite(d15_sigma) & (sigmas15 >= left) & (sigmas15 <= right)
# 置信检验 (稳健统计)
d10_med, d10_lo, d10_hi, rel_d10_med_pct = check(d10_sigma)
d15_med, d15_lo, d15_hi, rel_d15_med_pct = check(d15_sigma)
d_med_ref = np.median([d10_med, d15_med])
diff_pct = abs(d10_med - d15_med) / max(abs(d_med_ref), 1e-12) * 100.0

f15 = interp1d(sigmas15[m15], d15_sigma[m15], kind='linear', bounds_error=False,
               fill_value=np.nan)
x_common = sigmas10[m10]
y10_common = d10_sigma[m10] * 1e4
y15_on_10 = f15(x_common) * 1e4

print("[滑窗FFT]:")
print(f"S(10°) 中位数   {np.median(Ss10):.6f} cm, Δ   {1/np.median(Ss10):.2f} cm-1")
print(f"S(15°) 中位数   {np.median(Ss15):.6f} cm, Δ   {1/np.median(Ss15):.2f} cm-1")
print("在S=2 , 3 ...附近均未发现明显亮纹, 认为没有发生多光束干涉")

```